

LLM explainability

Alessandra Ciciarelli

December 2025

1 Fine tuning dataset

To fine-tune LLaMA 3 8B for answer generation and provenance explanation, we construct a dataset where each training example is represented as a structured `input,output` pair. The input consists of a natural language question and its corresponding SQL query, while the output is a JSON object containing both the query answers and their provenance, expressed as witness sets. Each answer is explicitly paired with a single witness set.

An example of a training instance is reported below:

```
{
  "instruction": "Return ONLY the JSON output. Each answer
must have exactly one corresponding witness set in the
'why' field, formatted as {{table_row}}.",
  "input": {
    "question": "What are the dates and points for each
constructor standing,
along with their corresponding subsequent standings, for the
28 most recent races?"
  },
  "output": {
    "results": [
      {
        "answer": { "date": "2008-03-16 04:30:00", "points": "14" },
        "why": "{{constructor_standings_1,races_1}}"
      },
      {
        "answer": { "date": "2008-03-16 04:30:00", "points": "8" },
        "why": "{{constructor_standings_2,races_2}}"
      }
    ]
  }
}
```

2 Curriculum Learning

Curriculum learning (CL) is a training strategy that trains a model from easier data to harder data, imitating learning order in human curricula. In the paper *A Survey on Curriculum Learning*?, training examples are ordered according to an explicit difficulty measure, such as the number of joins, self-joins, and output tuples. The model is first trained on simpler instances and progressively exposed to more complex queries, allowing it to learn the rigid output format and the semantics of witness sets before handling harder relational patterns.

Algorithm 1: Baby Step Training Scheduler ?

Input: D : training dataset; C : difficulty measurer

Output: M^* : trained model

$D' \leftarrow \text{sort}(D, C)$;

$\{D_1, D_2, \dots, D_k\} \leftarrow D'$ such that $C(d_a) < C(d_b)$, $d_a \in D_i$, $d_b \in D_j$, $\forall i < j$;

$D_{\text{train}} \leftarrow \emptyset$;

for $s = 1$ **to** k **do**

$D_{\text{train}} \leftarrow D_{\text{train}} \cup D_s$;

while *not converged for p epochs* **do**

 train(M , D_{train});

3 17/12/2025 meeting

Current Status

- Three datasets have been generated for model fine-tuning, namely *RELF1*, *RELSTACK*, and *TPC-H*, each consisting of 2000 SQL queries produced using *SQLSmith*.
- For each query, the result set and a corresponding *why*-provenance are available; however, the provenance information is currently represented as unstructured strings, limiting its usability for systematic processing and learning.
- The generated queries are not yet categorized according to their structural or semantic complexity (e.g., joins, aggregates, or set operations), preventing controlled evaluation across different query classes.
- The query generator currently excludes negation operators, as the *EXCEPT* clause in *SQLSmith* has been disabled.
- The fine-tuning datasets do not yet include the full relational context (i.e., all relevant tuples and attributes) required to answer each query in isolation.

Planned Work and Motivation

- **Dataset size characterization.** We will compute the effective size of each dataset by considering not only the number of queries, but also the average number of tuples in query results and the average size of the *why*-provenance. This will provide a more realistic estimate of dataset complexity and learning signal, enabling fairer comparisons across datasets and query types.
- **Structured provenance representation.** The *why*-provenance will be converted from a string-based format to a structured JSON representation based on lists of lists. This structured encoding is essential to support automated manipulation, facilitate supervision during fine-tuning, and enable future provenance-based reasoning tasks.
- **Query categorization by structural features.** The query generation and sampling process will be extended to annotate queries according to key characteristics, including:
 - number of joins,
 - presence of set operations (UNION, INTERSECTION),
 - use of aggregate functions (binary indicator and count),
 - presence of negation.

This categorization will allow controlled experimentation and evaluation, as well as targeted training on queries of increasing structural complexity.

- **Support for negation.** To include negation patterns, the `EXCEPT` clause in SQLSmith will be re-enabled. Negation is a fundamental relational operator and is known to increase query difficulty; incorporating it is therefore crucial for assessing the robustness of the proposed approach.
- **Specialized fine-tuning by query type.** Separate fine-tuning runs will be performed for different query categories (e.g., single-join, multi-join, aggregate queries), with model checkpoints stored after each training phase. This will enable an analysis of how query complexity affects learning dynamics and model performance.
- **Context-complete fine-tuning data.** Each fine-tuning example will be augmented with the full relational context required to answer the query, including all relevant rows and attributes. Providing complete context is necessary to prevent implicit information leakage and to ensure that the model’s predictions are grounded solely in the supplied data.
- **Query quality refinement.** Duplicate attributes in the `SELECT` clause will be removed during query generation to improve query quality and avoid introducing redundant or misleading supervision signals.
- **Model fine-tuning.** Once the datasets are finalized, supervised fine-tuning will be performed to adapt the model to structured query answering with provenance-aware explanations.
- **Model distillation.** After fine-tuning, knowledge distillation techniques will be explored to transfer the acquired capabilities to smaller or more efficient models, improving deployability while preserving performance.
- **Provenance-to-Natural-Language translation.** Finally, we will design and implement a Prov-to-NL translation component aimed at generating faithful and interpretable natural language explanations from structured provenance, improving the transparency and usability of the system.

Updates on work

- **Metadata added for query type classification** The metadata for classifying query based on their complexity have been added. For example:

```
-- meta {"num_joins":1,"num_aggregates":0,
"has_union":true,
"has_intersect":false,
"has_negation":false}

-- meta {"num_joins":2,
"num_aggregates":0,
"has_union":true,
"has_intersect":false,
"has_negation":false}
```

- **Set operations in SQLSmith.** Set operations were enabled in the query generator to increase query diversity. In particular, `UNION` and `EXCEPT` are now supported, while `INTERSECT` remains disabled due to limited support in provsql.
- **Dataset statistics.** Dataset statistics are now computed from the final JSONL files, including the number of successful and failed queries, the total number of answer tuples, and detailed provenance size. For example, for the dataset *relf1*:

```

"db": "relf1",
"input_sql": "set_queries/queries_relf1_mixedwithset.sql",
"output_jsonl": "queries_with_prov/relf1_mixedwithset_prov.jsonl",
"generated_at": "2026-01-07T12:34:56",
"total_lines_in_jsonl": 2000,
"ok_queries": 1999,
"failed_queries": 1,
"total_answer_tuples": 47785,
"total_why_sets": 54752,
"total_why_items": 83820,
"avg_answers_per_query": 23.904452226113058,
"avg_why_items_per_query": 41.93096548274137,
"avg_why_items_per_tuple": 1.754106937323428,
"effective_size_proxy": 133604

```

- **Structured provenance representation.** The *why*-provenance is no longer stored as an unstructured string, but is represented as a normalized list of lists in JSON, where each inner list corresponds to a witness set. This format facilitates downstream processing and model supervision.
- **SQL to NL** The sql to nl json files have been produced for the three datasets: relf1, relstack and tpch. The format of each row is:

```

{
  "id": 1999,
  "meta": {
    "num_joins": 0,
    "num_aggregates": 0,
    "has_union": false,
    "has_intersect": false,
    "has_negation": false
  },
  "sql": <sql_query>,
  "nl": <generated_nl_query>
},

```

4 Meeting 07/01/2026

Meeting Summary (Prompt and Dataset Refinement)

- The fine-tuning prompt will be revised to exclude SQL from the input and to provide more precise and correct instructions to the model.
- A separate set of queries will be created for validation, ensuring they are different from those used during training.
- Instead of multiple datasets, a single dataset will be used for the fine tuning, enriched with explicit annotations capturing query complexity. This because we need one model able to handle different type of queries.
- Negative pairs will be added to the fine-tuning dataset by perturbing positive examples, e.g., by adding or removing tuples or sets, to improve the model's discriminative capabilities.